

Write to me if you couldn't locate the answer to your question in the link provided, or were unsatisfied with the quality or quantum of information presented, and I will try to respond with the profound knowledge locked up in my highly-evolved brain. A riveting quality of my mind is that it spontaneously goes into a meditative state half-way through a negative feedback, and instantly rejects the hypothesis that I may be too dumb to understand that I am a bad writer. My superior brain and eyes then work in sync to dismiss any dissenting remarks or profanities. This enables me to rant on with the technical mumbo-jumbo that you now hold in your hand. If you sent negative feedback about my previous article to the editor, and did not receive a reply, you now know why!

Getting acquainted

Lisp, whose name is an acronym for LISt Processing, was designed to provide symbol-manipulating capabilities to attack programming problems such as symbolic differentiation and the integration of algebraic expressions. Despite its inception as a mathematical formalism, Lisp is a practical programming language. The basic elements of Lisp include its primary data structure, called the s-expression. This also includes the Lisp interpreter, which is the heart of any Lisp system, and is basically a machine that carries out processes described in the Lisp language. The Lisp interpreter performs computations on s-expressions through a process called evaluation. Since its earliest days, Lisp implementations have been variously interpreted or compiled, and often both. No modern commercial version of Lisp is without a compiler. The fact that modern Lisp variants often come with an interpreter as well, is simply a convenience for some implementations, to encourage late-binding semantics and promote flexibility, including interactive debugging.

The prompt

I assume that you have CLISP on your machine (please read Part I of the Lisp article in the June 2011 edition of *Linux For You* for instructions on how to install and use CLISP). The exact command to start a Lisp process may vary from one OS to another. Any Lisp system will include an interactive front-end called the *toplevel*. You type Lisp expressions into the *toplevel*, and the system displays their values. Lisp usually displays a prompt to tell you that it is waiting for you to type something. Many implementations of Common Lisp use `>` (the 'greater-than' symbol) as the *toplevel* prompt; some may use an entirely different prompt, and others may issue no prompts at all.

S-expressions

A Lisp command begins with a parenthesis. Next, specify the name of an operation that you would like to perform. Then give the arguments you want to use. Finish the whole thing off with a final parenthesis. For example, if

you want to add two numbers together, type something like the following code:

```
> (+ 1 2)
3
```

In this example, you only typed the expression in parenthesis; the prompt and the answer were output by Lisp. In many implementations of Lisp, the user types a carriage return to signal that the command is complete. Other implementations will go to work as soon as they see the closing parenthesis. Also, some implementations will output additional blank lines along with the result.

In the expression `(+ 1 2)`, the `+` is called the addition operator, and the numbers 1 and 2 are called the arguments. In other programming language(s), this expression would be written as `1 + 2`, but in Lisp the `+` operator is put first, followed by the arguments, with the whole expression enclosed in a pair of parenthesis. This is called prefix notation, because the operator comes first. Lisp's prefix notation offers convenience to programmers. For example, if you want to add five numbers together, in ordinary notation (and programming language) you have to use `+` four times:

```
1 + 2 + 3 + 4 + 5
```

...while in Lisp, you just add other arguments:

```
> (+ 1 2 3 4 5)
15
```

Commands like these are called s-expressions, which stands for symbolic expressions. This is a very general term, applicable to just about anything one can say in Lisp. The basic elements of s-expressions are lists and atoms. Lists are delimited by parentheses, and can contain any number of elements separated by white-space. Atoms are everything else. The elements of lists are themselves s-expressions (in other words, atoms or nested lists). More about lists and atoms will follow in the next article. Comments—which, technically speaking, are not s-expressions—start with a semi-colon, extend to the end of a line, and are essentially treated like white-space.

```
> ;demo of multiplication operator
(* 3 7)
21
```

Because operators can take a varying number of arguments, we need parentheses to show where an expression begins and ends. Expressions can be nested—that is, the arguments in an expression may themselves be complex expressions. Suppose you want to compute